BÁO CÁO LAB-03

Debug ứng dụng và đánh giá hiệu năng Linux nhúng

Linux 5.15 · ARMv7 Cortex-A9 · QEMU · GDB · Valgrind · strace · cyclictest

1. Thông tin sinh viên

Họ và tên	Nguyễn Cao Phát	**MSSV**	SE201044
Môn học	Embedded Linux Systems		
Ngày thực hiện	04/11/2026		

2. Mục tiêu và cách làm

Trong LAB-03, em kiểm tra cách debug chương trình ARM, tìm lỗi rò rỉ bộ nhớ và quan sát syscall/độ trễ. Bài làm dùng lại kernel và DTB của LAB-01, đồng thời tạo một bản initramfs riêng từ LAB-02 rồi thêm chương trình LAB-03 và ARM gdbserver. Các file build gốc của hai bài trước không bị sửa.

Hạng mục	Môi trường hoặc công cụ
Target	ARMv7-A Cortex-A9 trên QEMU vexpress-a9; Linux 5.15.0
Debug	GDB multiarch trên host kết nối ARM gdbserver trong guest; có thêm bài chạy QEMU user-mode
Bộ nhớ	Valgrind Memcheck trên bản host; bản ARM được chạy trong QEMU
Syscall và CPU	strace, perf stat/record
Độ trễ	cyclictest trên host Linux 7.0.0-42-generic; PREEMPT_DYNAMIC

3. Tích hợp LAB-01 và LAB-02

Script kiểm tra xác nhận kernel có phiên bản 5.15.0, bật module, LAB-01 và LAB-02 dùng cùng zImage/DTB, và initramfs LAB-02 có driver đã build cho kernel đó. Khi boot QEMU bằng initramfs mới, các

bài test thiết bị ký tự và procfs/sysfs của LAB-02 vẫn chạy được; ứng dụng ARM của LAB-03 cũng chạy thành công.

```
LAB-01 kernel/DTB booted the LAB-03 initramfs: PASS
LAB-02 character-device test: PASS
LAB-02 procfs/sysfs test: PASS
LAB-03 ARM applications: PASS
ARM gdbserver listens on guest port 1234: PASS
```

4. Debug chương trình ARM

Ứng dụng debug được build có symbol và tắt tối ưu hóa để dễ theo dõi. Em tách các trường hợp lỗi thành null pointer, truy cập vượt mảng, use-after-free và signed integer overflow. Với trường hợp mảng, vòng lặp dùng điều kiện `i <= len`, nên lần lặp cuối truy cập chỉ số `len` trong khi chỉ số hợp lệ là 0 đến `len-1`.

Em kết nối host GDB đến gdbserver trong guest qua cổng QEMU forward. Đặt breakpoint có điều kiện ngay trong vòng lặp giúp dừng tại `i == len`; nếu dừng ở đầu hàm thì biến `i` chưa nằm trong phạm vi truy cập.

Quan sát	Kết quả của phiên chạy
Điểm dừng	`process_data()`, tại dòng đọc `data[i]`, với `i=len=5`.
Đọc vượt mảng	GDB hiển thị `data[5]=418247424` ở lần chạy được ghi log. Đây là giá trị ngoài vùng hợp lệ, không ổn định và không nên xem là kết quả cố định.
Ngăn xếp	`process_data()` được gọi từ `main()`.
Thanh ghi ARM	`pc=0x10480`, `sp=0x7efffcd8`, `lr=0x10637` tại thời điểm dừng.
Các trường hợp khác	Các log GDB riêng cho null pointer, use-after-free và overflow được lưu trong thư mục logs.

Với lỗi tràn số nguyên có dấu, kết quả phụ thuộc hành vi không xác định của C; em không xem một giá trị quan sát được là quy tắc wraparound.

5. Kiểm tra bộ nhớ và syscall

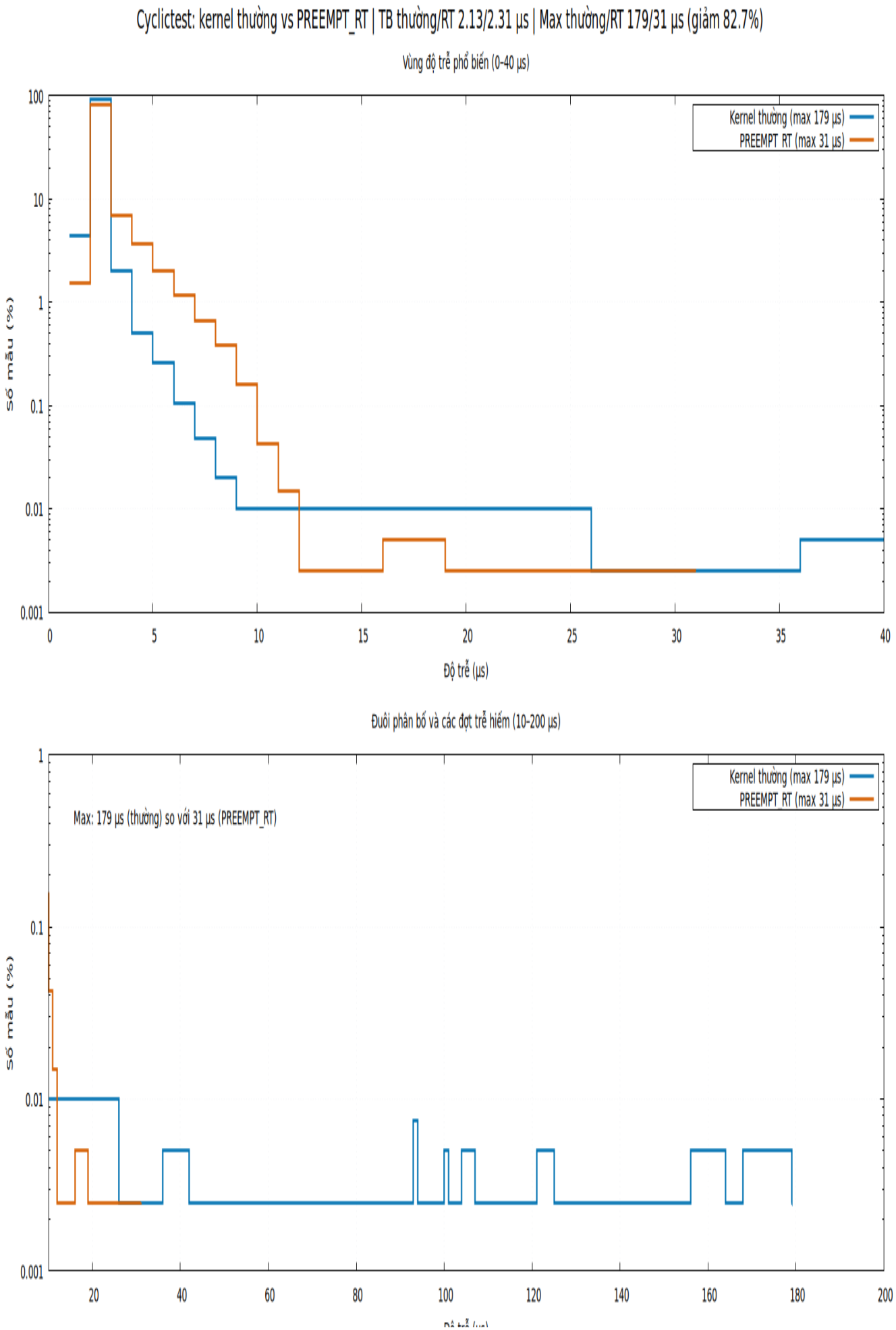
Phép thử	Kết quả ghi nhận

Valgrind, chương trình có chủ ý rò rỉ	Còn 16.900 byte trong 250 block; tất cả được phân loại definitely lost.
Valgrind, bản đã sửa	351 lần cấp phát, 351 lần giải phóng; còn 0 byte và 0 lỗi.
strace	Ghi nhận 119 syscall, 1 lỗi và tổng thời gian syscall khoảng 0,671 ms trong lần chạy này.
perf	Không lấy được mẫu CPU đáng tin cậy: bộ đếm không hỗ trợ hoặc bị chính sách host chặn, `perf_event_paranoid=4`.

Valgrind chạy trên bản x86-64 trong container công cụ; kết quả dùng để tìm vị trí cấp phát bị rò rỉ, không phải phép đo dung lượng RAM chính xác trên ARM. `/proc/<pid>/status` cho RSS tại một thời điểm, nhưng không đủ để kết luận có hay không có leak.

6. Đo độ trễ và giới hạn

Em chạy cyclictest trên host, không chạy trong QEMU, với 4 thread, chu kỳ 1.000 µs, ưu tiên SCHED_FIFO 80 và 10.000 vòng mỗi thread. Hai lần đo dùng cùng thông số trên kernel thường PREEMPT_DYNAMIC và kernel PREEMPT_RT. Histogram gộp 40.000 mẫu cho mỗi lần chạy. Trung bình tính từ histogram lần lượt là 2,13 µs và 2,31 µs; độ trễ lớn nhất lần lượt là 179 µs và 31 µs. Như vậy, trong hai lần đo này, max của PREEMPT_RT thấp hơn khoảng 82,7%, nhưng trung bình không thấp hơn. Đây là kết quả quan sát trong một lần chạy cho mỗi kernel, không khẳng định hiệu năng cho mọi tải hoặc phần cứng.


Cyclictest: kernel thường vs PREEMPT_RT | TB thường/RT 2.13/2.31 µs | Max thường/RT 179/31 µs (giảm 82.7%)
Vùng độ trễ phổ biến (0-40 µs)
Kernel thường (max 179 µs)
PREEMPT_RT (max 31 µs)
Số mẫu (%)
Độ trễ (µs)
Đuôi phân bố và các đợt trễ hiếm (10-200 µs)
Max: 179 µs (thường) so với 31 µs (PREEMPT_RT)

Hình 1. So sánh phân bố độ trễ đã chuẩn hóa theo phần trăm tổng mẫu. Biểu đồ phóng to vùng 0-40 µs và tách riêng phần đuôi 10-200 µs để thấy các mẫu trễ hiếm.

Biểu đồ histogram được tạo từ kết quả thực tế bằng script `plot_latency.gnuplot`. Em chưa đo lặp nhiều lần hoặc tạo tải nền được kiểm soát, nên so sánh này có tính tham khảo. perf chưa cho phép xác định hotspot cụ thể do giới hạn quyền đo trên host.

7. Nhận xét và kết luận

Qua bài này em đã thực hành lần theo lỗi bằng breakpoint, biến, backtrace và thanh ghi; dùng Valgrind để so sánh bản lỗi với bản đã sửa; và dùng strace/cyclictest để quan sát hành vi hệ thống. Các phép đo phụ thuộc môi trường nên em chỉ ghi số liệu có log xác nhận. Phần cần làm tiếp nếu có môi trường phù hợp là đo lại perf và chạy đối chứng trên kernel PREEMPT_RT.

Log tương thích, GDB, Valgrind, strace, perf và cyclictest được lưu trong thư mục `logs/` của gói LAB-03.